doekit manual and standard operating procedure

Purpose. This document explains how to plan, run, analyse and optimise an experiment with doekit, and how to choose the right design for your application.

Who it's for. Researchers who want to use design of experiments (DOE) for any process, such as bioprinting, formulation or synthesis. You don't need a statistics or programming background to follow it.

Version. Written for doekit 0.1.

Contents

1. DOE in five minutes
2. Procedure
3. Choosing a design
4. Is composition a mixture?
5. Reading your results
6. Worked example: a hydrogel bioink
7. Using the web app
8. Using the scripts and config files
9. Troubleshooting
10. What doekit doesn't do
11. Methods behind each step
12. How doekit was checked
13. References

1. DOE in five minutes

The usual way to optimise a process is to change one factor at a time: find the best pressure, fix it, then find the best speed, and so on. This has two problems. It misses **interactions**, where the best pressure depends on the speed. And it uses each run to learn about only one factor.

DOE changes several factors at once, in a planned pattern. Each run then tells you something about every factor, so you learn more from fewer runs. Afterwards, a statistical model separates out each factor's effect.

Term	Meaning	Bioprinting example
Factor	Something you deliberately change	Pressure, GelMA concentration
Level	A setting of a factor	40 kPa and 120 kPa
Response	Something you measure	Filament width, cell viability
Run	One experiment at one combination of settings	One printed construct
Effect	How much a response changes when a factor changes	Width rises 220 μm from low to high pressure
Interaction	When one factor's effect depends on another factor	Pressure matters more at low speed
Curvature	When a response bends rather than following a straight line	Viability peaks at a middle temperature
Model	An equation linking factors to a response	Width = 400 + 110 × pressure − 75 × speed
Noise	Variation between runs at identical settings	Differences between repeat prints

Three principles protect your conclusions, and doekit builds them in:

- **Randomise.** Do runs in random order. Otherwise, anything that drifts during the experiment, such as ageing material or a warming room, could be mistaken for a factor's effect.
- **Replicate.** Repeat some runs completely, to measure noise. A real replicate starts again from preparing the sample. Measuring one sample three times is not a replicate, because it only shows the noise of the measurement.
- **Block.** If runs must be done in groups, such as over several days or from different bioink batches, say so. Differences between the groups can then be separated from the factors' effects.

Most projects go through DOE in stages, because it's more efficient than one big experiment:

1. **Screen** many possible factors with few runs, to find the ones that matter.
2. **Optimise** those few factors with a response surface design, to find the best settings.
3. **Confirm** the prediction with new runs.

2. Procedure

Follow these steps in order. Step 1 happens away from the computer, and it's the one that most affects whether the experiment succeeds.

Step 1: Plan

Write down the following before opening doekit. Talk it through with someone who knows the process.

□ **Objective.** What decision will this experiment inform? For example, "choose a blend and print settings for a cartilage construct."

□ **Responses.** What will you measure, how, and what value is good? Check that the measurement is reliable first. A noisy measurement can hide any effect.

□ **Factors.** List everything that might matter. Decide which to vary, which to hold fixed, and which you can't control (record those).

□ **Ranges.** Choose low and high settings for each factor. They should be wide enough to show an effect, but narrow enough that every run still gives a usable result. Too narrow is the most common mistake. Run a few trial prints at the extremes before starting.

□ **Budget.** How many runs can you afford, in material, cells and time?

□ **Groups.** Will the runs span several days or batches? If so, plan to use blocks.

□ **Failure.** What happens if an extreme setting doesn't print at all? If you expect that, choose a design that avoids extremes, or narrow the range.

Step 2: Define the study

Enter your factors and responses in the app (step 1 on screen), or copy a template from `configs/templates/` if you're using scripts. Give each response a goal: maximise, minimise, target or stay in a range.

Step 3: Choose a design

Use section 3. The app can suggest designs from your factors, objective and budget, and shows real run counts for each option. The methods are listed under Creating the design.

Step 4: Check the design before you start

This costs nothing, and it's the last chance to change the plan. Look at the following (the calculations are under Judging a design):

Check	Good sign	If not
Spare runs	At least 3 to 5 more runs than model terms	Add runs, or simplify the model
Replicates	At least 3	Add centre points or replicate runs
Lowest power	At least 0.8 for the effects you care about	Add runs, or accept that smaller effects could be missed
Warnings	None you can't explain	Read each one, and act on it or note why not
The run sheet itself	Every run looks physically possible	Narrow the range, add a constraint, or use a design that avoids extremes

Step 5: Run the experiments

- Do the runs **in the order of the `run` column**. The order is random on purpose. If a factor is truly hard to change, see section 10 before changing the order.
- Keep everything you aren't testing the same: operator, equipment, material lot and protocol. Note anything that does change.
- Don't drop or redo a run because the result looks odd. Unexpected results are often the most informative.
- If a run fails, leave its result blank and write the reason in the `notes` column.

Step 6: Enter the results

Type results directly into the app, or fill in the run sheet in Excel and upload it. European Excel settings (semicolons and decimal commas) are handled automatically. If the settings you actually achieved differ from the plan, such as a measured concentration, enter what you achieved: the analysis uses whatever's in the sheet.

Step 7: Analyse

For each response, read the **verdict** first, then use section 5 to work out what to do about anything flagged. Don't move on with a model the checks don't support. The methods and thresholds are under Fitting the model and Checking the model.

Step 8: Find the best settings

doekit searches for the settings that best meet all your goals at once, and predicts each response with a range. Look at the **sweet spot** plot as well as the single best point. A wide sweet spot means the process tolerates small errors, which often matters more than a slightly better peak. The methods are under Finding the best settings.

Step 9: Confirm

This step isn't optional. The best settings are a prediction, not a measurement.

1. Prepare the best settings and do **at least 3 independent confirmation runs**.
2. Compare the results with the prediction range doekit gave for a new run.
    - **Inside the range:** the model is confirmed. Use these settings.
    - **Outside the range:** something is missing from the model. Don't use the prediction. Run a smaller follow-up experiment around this region. You can add those runs to this study rather than starting again.

Step 10: Keep your records

Keep the following together, and store them with your lab notebook entry:

- The study config file. Together with its seed, this recreates the exact design.
- The completed run sheet, including notes.
- The reports and figures, and the results of the confirmation runs.

When you report a study, state the design type, number of runs and replicates, the final model, R^2, adjusted R^2 and predicted R^2, the lack-of-fit result, and the confirmation results. Writing up your methods has

wording you can adapt.

3. Choosing a design

Answer these questions in order. The app's **Suggest designs** button follows the same logic, and shows the real run count and quality of each option.

Question 1: What's your objective?

Objective	Typical situation	Go to
Screen	Many factors (usually 5 or more), and you don't know which matter	Question 2
Optimise	A few key factors (usually 2 to 5), and you want the best settings	Question 3
Explore	Behaviour is unknown or complex, or the data is for machine learning	Latin hypercube or Sobol

Question 2: Screening. How many factors do you have?

Factors	Recommended design	Why
2 to 4	Full factorial with centre points	You can afford every combination
4 to 11	Fractional factorial, resolution IV	Main effects stay clear of two-factor interactions
5 to 8, if you can afford it	Fractional factorial, resolution V	Main effects and two-factor interactions are all separated
6 or more, with very few runs	Plackett-Burman	The fewest runs, for main effects only
Any, with categorical factors of 3 or more levels, or constraints	D-optimal with a linear model, or generalized subset	Two-level designs don't fit

Question 3: Optimising. What kind of factors do you have?

Your factors	Recommended design	Why
Composition and process together	**I-optimal**	Classical designs can't handle a blend together with process factors
Composition only, wide limits	Simplex centroid or simplex lattice	The standard blend designs
Composition only, tight limits	I-optimal or extreme vertices	Limits cut corners off the blend region
Any categorical factors, or constraints	I-optimal	Classical response surface designs don't fit
1 numeric factor	Full factorial, 5 levels, repeated	Shows the shape of the curve
2 numeric factors	Central composite, face-centred	Efficient, and it stays within your limits
3 to 5 numeric factors, all corners possible	Central composite, face-centred	Describes the whole region
3 to 5 numeric factors, extremes would fail	**Box-Behnken**	Never sets every factor to an extreme at once
6 or more numeric factors	Screen first, or I-optimal	Otherwise the number of runs gets very large
A fixed run budget that doesn't fit the above	I-optimal	Works with any number of runs above the minimum

What each design does

Design	Good for	Watch out for
Full factorial	Few factors, and measuring every interaction	Doubles in size with each extra factor
Fractional factorial	Screening with some interaction information	Some effects can't be told apart. The report lists which
Plackett-Burman	Screening many factors very cheaply	Main effects are partly mixed with interactions
Taguchi orthogonal array	Balanced multi-level screening	Assumes interactions are small. doekit analyses it by regression rather than Taguchi's signal-to-noise method
Generalized subset	Several factors with more than 2 levels	Main effects only, unless you use a higher-order model that the design supports
Central composite	The standard for fitting curvature	The circumscribed variant goes beyond your limits
Box-Behnken	Avoiding extreme combinations	Says little about the corners, so don't predict there
Doehlert	Designs you may later extend in one direction	Factors get different numbers of levels
Simplex lattice or centroid	Blends with wide limits	Includes pure components, which may not be practical
Extreme vertices	Blends with tight limits	The number of corners can grow quickly
Crossed	Showing every blend at every process setting	Needs many runs
D-optimal	Precise coefficients, for understanding effects, with any factors or budget	Depends on the model you specify, and favours the edges of the region
I-optimal	Precise predictions, for finding the best settings, with any factors or budget	Depends on the model you specify
Latin hypercube or Sobol	Unknown behaviour, or data for machine learning	Less efficient if a quadratic model is enough. Replicates need adding separately

D-optimal or I-optimal?

Both pick runs by computer, for the model you specify. **Use I-optimal if your goal is finding the best settings**, because it makes predictions most precise across the whole region. **Use D-optimal if your goal is understanding which effects are real**, because it makes the coefficients most precise.

How many runs will I need?

It depends on what you're trying to learn, so there's a table for each purpose. These counts come from doekit. A blank means that design isn't available for that number of factors.

To find out which factors matter (screening). These designs test each factor at two settings, so they can't locate an optimum. Each count includes 3 repeated runs at the centre of the ranges, which measure noise and can reveal curvature.

Factors	Full factorial (2 levels)	Fractional, resolution IV	Fractional, resolution V	Plackett-Burman
2	7			7
3	11			7
4	19	11		11
5	35	19	19	11
6	67	19	35	11
7	131	19	67	11
8	259	19	67	15

To find the best settings (optimising). These designs use at least three settings per factor, so they can fit a quadratic model, which needs at least as many runs as it has terms.

Factors	Terms in a quadratic model	Full factorial (3 levels)	Box-Behnken	Central composite	I-optimal
2	6	9		12	14
3	10	27	15	18	18
4	15	81	27	28	23
5	21	243	43	46	29
6	28	729	63	80	36
7	36	2187	87	146	44
8	45	Too many	115	276	53

Box-Behnken counts include 3 centre points, and central composite counts include 4. I-optimal counts include 4 extra runs to check the model and 4 replicates. The 3-level full factorial has no repeats, so you'd need to add some to measure noise.

Don't compare numbers between the two tables. A two-level factorial can need fewer runs than a Box-Behnken design because it answers a simpler question: which factors matter, not where the best settings are.

For blends of 3, 4 and 5 components, a simplex lattice of degree 2 needs 6, 10 and 15 runs, and a simplex centroid needs 7, 15 and 31, before adding check points or replicates. A blend with process factors usually needs an optimal design: the worked example in section 6 uses 32 runs for 3 components and 3 process settings.

Is that enough to detect the effects I care about? The power figure answers this. It's the chance of detecting a low-to-high change of a given size, measured as a multiple of the noise. Enter the smallest effect worth detecting, and aim for a power of at least 0.8. If you don't know your noise, a small pilot of repeated runs at one setting will tell you, and it's often worth doing anyway.

Examples

Situation	Choice
You have one established bioink, and want the best pressure, speed and temperature. High pressure with high speed clogs the nozzle.	**Box-Behnken**, 3 factors, 15 runs
You have eight candidate additives and settings, and budget for about 15 prints.	**Plackett-Burman** with centre points, 15 runs, then optimise the important ones
You want to optimise a three-polymer blend and its printing settings together.	**I-optimal**, mixture and process model, about 30 runs
You're comparing three crosslinkers while optimising concentration and time.	**I-optimal**, because of the categorical factor
You're optimising two jetting settings, such as pressure and valve open time.	**Central composite** (face-centred), or a 3 × 3 full factorial
You're optimising five melt electrowriting settings where all-extreme runs are risky.	**Box-Behnken**, 40 runs plus 6 centre points
You're collecting data for a machine learning model of a complex process.	**Latin hypercube**, at least 10 runs per factor

Adding more runs later

You don't have to get everything right in one experiment. Working in stages is often the most efficient way to run a study: learn something, then decide what to do next. doekit can add runs to a design you've already started, and your existing runs and results don't change.

Your existing runs stay fixed, and the optimal design search picks the new runs that add the most to them, for the model you want to fit. Common reasons to add runs:

Situation	What to set
Screening found the important factors, and now you want to find the best settings.	Choose the model with curves and interactions, and the search adds the runs that make it possible. All your factors stay in the study. To drop the ones that didn't matter, set up a new study instead (see below)
The model doesn't fit well, or the effects are less certain than you need.	Keep the same model, add runs, and aim for better estimates or better predictions
The best settings are near the edge of a range you tested, and you want to go further.	Widen that factor's limits in step 1, then add runs. The new runs can then go up to the new limits
You want more detail around promising settings.	Keep the new runs inside part of the range. Your earlier runs still count towards the model
You didn't include repeats, so you don't know your noise.	Set some of the new runs to repeat earlier ones

Doing the new runs in a separate session. Later experiments often differ a little from earlier ones, such as a new batch of material, another day or another operator. If that applies, leave the session option on. The new runs then go in their own **block**, so a difference between sessions isn't mistaken for a factor effect. Repeating 2 or more earlier runs in the new session also shows how well the two sessions match.

Failed runs. A run with no result hasn't told you anything, so it isn't counted when choosing the new runs. It stays in your run sheet, along with your notes.

When to set up a new study instead. Start again if you've removed factors, moved to a very different region, or changed something the factors don't capture, such as a new supplier or a different printer. Runs from the old conditions may not apply to the new ones, and combining them could mislead you.

To add runs, see section 7 for the app or section 8 for the scripts.

4. Is composition a mixture?

This decision matters a lot for hydrogels and other formulations, because it changes which designs and models apply. There are two legitimate ways to describe composition.

Option A: independent concentrations (continuous factors). Each ingredient is set on its own, and water or solvent makes up the rest. For example, GelMA 5 to 15% w/v and alginate 0.5 to 3% w/v. Changing GelMA doesn't force you to change alginate.

Option B: a blend (mixture components). The ingredients are shares of a fixed total, so increasing one must decrease another. For example, GelMA, alginate and nanocellulose as fractions of the total polymer, adding up to 1.

Ask yourself	If yes	If no
Must the proportions add up to a fixed total?	Mixture	Continuous
Is the question about *ratios* between ingredients, at a fixed total amount?	Mixture	Continuous
Is a solvent the large majority, and free to make up the difference?	Continuous is usually simpler	Mixture
Could every ingredient reach its high setting at the same time?	Continuous is fine	Mixture, or add a constraint

Both can be combined. To study ratios and total amount separately, describe the ratios as mixture components and add total polymer concentration as a continuous factor.

A rule of thumb for hydrogels. If you think "I'll try 5, 10 and 15% GelMA with 1, 2 and 3% alginate", use continuous factors. If you think "I'll vary the ratio of the polymers in the blend", use mixture components.

Why it matters: with mixture components, you can't say "the effect of raising GelMA with everything else fixed", because something else must fall. Mixture models account for this. They have no intercept, and each coefficient describes a blend. Compare the component coefficients with each other, not with zero. Positive blend terms, like `gelma:alginate`, mean synergy, and negative ones mean antagonism.

If you think a factor combination can't be made or printed, add a **constraint**, such as `gelma_wt + alginate_wt` with a maximum of 16, and use an optimal design. See `configs/examples/hydrogel_concentrations_simulated.yaml` for this, and `configs/examples/hydrogel_bioink_simulated.yaml` for a mixture.

5. Reading your results

Start with the verdict, which turns the statistics into plain statements. Here's what each check means, and what to do when it flags something. The exact tests and thresholds are under Checking the model.

Check	What it asks	If flagged
Overall model	Does the model explain more than noise would?	The factors may have little effect in these ranges. Try wider ranges, or reduce the noise
Predictive power	How well does the model predict runs it hasn't seen?	The model is probably fitting noise. Turn on model reduction, and look for influential runs
Lack of fit	Does the model miss a real pattern?	Try a transform, or a higher-order model. Be careful with predictions until it's resolved
Residual normality	Are the leftover errors roughly bell-shaped?	Look for outliers, and try a transform
Constant variance	Is the scatter the same at low and high values?	Try a log or square-root transform
Transform	Would a transformed response fit better?	Set the suggested transform, then compare
Run order	Did something drift during the experiment?	Check the lab notes for a cause, such as a material ageing. Repeat the affected runs if you can, and use blocks next time
Outliers	Does any run not fit the pattern?	Check the notes for that run. **Only remove a run if you can name a real cause**
Influential runs	Does one run strongly affect the model?	Verify that measurement. Consider repeating that run
Curvature	In a two-level design, do centre points differ from corners?	Straight lines aren't enough. Move on to a response surface design

The three R^2 values

- **R^2** is the share of the variation the model explains. It always rises as you add terms, so a high value alone means little.
- **Adjusted R^2** penalises extra terms.
- **Predicted R^2** measures how well the model predicts runs left out of the fit, so it's the most honest of the three. **It should be within about 0.2 of adjusted R^2.** A much lower or negative value means the model won't predict new runs well.

The figures

Figure	What to look for
What matters (Pareto)	Bars past the line are clear effects. Grey bars could be noise
Half-normal plot	Appears when no runs are spare. Points far from the line are real effects
Effect of each factor	The slope shows the effect's size, and the band shows its uncertainty. A flat line means no effect
Interactions	Parallel lines mean no interaction. The more they diverge, the stronger it is
Surfaces and contours	The shape of the response over two factors, with the others held fixed at the values in the title
Blend triangle	The response across the allowed blends. Each corner is that component at its highest possible share
Sweet spot	Dark areas meet all your goals. Wider is more robust
Model checks	Look for patterns. There shouldn't be any: no funnel, no curve, no trend over run order

Coefficients

- Coefficients are in **coded units**, where each process factor runs from −1 (low) to +1 (high). This lets you compare sizes directly, whatever the original units. Each coefficient is half the change from low to high.
- For a categorical factor, each coefficient is that level's difference from the average of all the levels.
- For a mixture, see section 4.

Model reduction

Turning on model reduction repeatedly removes the least significant term, while keeping the model hierarchical (a main effect stays while an interaction using it remains). This often improves predictions. But it's a tool, not a verdict: if you know from the science that a term matters, a high p-value only means this experiment couldn't detect it.

6. Worked example: a hydrogel bioink

> **Every number in this section is simulated.** The results come from made-up formulas in the config file, so that you can check the analysis against a known truth. They aren't real properties of these materials. Your own run may differ slightly with other software versions, or in the app if you pick a different number of runs.

Objective. Find the blend of GelMA, alginate and nanocellulose, and the print settings, that give a printable, stiff bioink with high cell viability.

Factors. The three polymers are shares of the total polymer content, so they're mixture components. Pressure, speed and nozzle temperature are continuous process factors.

Responses.

Response	Goal	Limits	Importance
Printability (Pr)	Target 1.0	0.85 to 1.15	3
Filament width (µm)	Target 400	300 to 500	2
Storage modulus (kPa)	Maximise	5 to 25	1
Cell viability (%)	Maximise	70 to 95	3

Design. Blend and process factors together call for an I-optimal design. A quadratic blend, quadratic process settings and blend × process interactions give 21 terms, plus 1 for the two bioink batches. The design uses 32 runs, of which 4 are replicates.

Checking the design. It leaves 10 spare runs, with typical prediction error of 0.76 × noise. One warning says some process terms have under 80% power for a 2 × noise change. That's acceptable here, but it's the kind of trade-off to think about before starting.

Results. After model reduction, all four models pass every check:

Response	R^2	Adjusted R^2	Predicted R^2	Lack of fit (p)
Printability	0.995	0.992	0.986	0.65
Filament width	0.994	0.989	0.980	0.75
Modulus (log scale)	0.851	0.799	0.719	0.89
Viability	0.985	0.973	0.951	0.88

Best settings. Overall desirability is 0.86, at 55% GelMA, 27% alginate, 18% nanocellulose, 61 kPa, 4 mm/s and 26 °C.

Confirmation. With a real material, you'd now do confirmation runs. Because this is a simulation, the true values at these settings can be calculated:

Response	Predicted	A new run should fall between	True value
Printability	1.00	0.96 and 1.04	1.01
Filament width (µm)	400	371 and 429	405
Modulus (kPa)	17.5	15.6 and 19.7	17.1
Viability (%)	88.4	85.1 and 91.6	89.2

All the true values fall inside the predicted ranges, so the workflow has found the answer the formulas built in.

Try it yourself. Load this example in the app, or see section 8 for the scripts. Then experiment: use fewer runs, raise the noise in the `simulation` section of the config, or change the goals, and watch how the conclusions change. It's a safe way to build intuition before using real materials.

7. Using the web app

If someone hosts the app for your group, open the address they give you. Otherwise, you can run it on your own computer.

Running it on your own computer

The app then runs entirely on your computer, so your data stays there and you don't depend on anyone else's machine. This has been tested on macOS. Windows should work, but hasn't been tested yet.

Set it up once:

1. Install Miniconda, if you don't already have conda.
2. Install Git, if you don't already have it. You don't need a Git or GitHub account. To check, open a terminal (on Windows, use Anaconda Prompt) and run `git --version`. If it isn't installed:
   - **On a Mac:** that command offers to install Apple's Command Line Tools, which include Git. Accept, and wait for it to finish.
   - **On Windows:** download the installer from git-scm.com and keep its default options. Then open a new Anaconda Prompt.
   - **If you can't install software on your computer:** run `conda install --override-channels -c conda-forge git`.
3. In the terminal, go to the folder where you want to keep the project, such as Documents. Then run:

```
git clone https://github.com/hagneeptatt/doe.git
cd doe
conda env create -f environment.yml
```

The last command can take a few minutes. If it says the `doe` environment already exists, run `conda activate doe` and then `pip install -e .` instead.

Make an icon to start it with a double-click. This is optional, and you only need to do it once. From the `doe` folder, run:

```
conda activate doe
python scripts/make_shortcut.py
```

This puts a **doekit** icon on your Desktop, and on Windows it also adds doekit to the Start menu. Double-click the icon to start doekit. After a few seconds it opens in your browser, at http://localhost:8510, and stays private to your computer. Clicking the icon again while doekit is running just opens it again. On a Mac, the icon stays in the Dock while doekit is running, so you can click it there too.

To stop doekit, choose **Close doekit** at the bottom of the sidebar. On a Mac, you can also right-click the icon in the Dock and choose **Quit**. If you only close the browser tab, doekit keeps running in the background. That does no harm, and the icon reopens it. If you move the `doe` folder or set it up again, run the command again. The icon has been tested on macOS, but not yet on Windows or Linux.

Or start it from a terminal each time, from the `doe` folder:

```
conda activate doe
streamlit run app/streamlit_app.py --server.address localhost
```

Then go to http://localhost:8501 in your browser. `--server.address localhost` keeps it private to your computer. Without it, other people on your network may be able to open the app. To stop it, press Ctrl+C in the terminal.

To update to a new version, stop the app, then run these from the `doe` folder and start it again. Your icon keeps working. Your saved studies and your own config files stay as they are.

```
git pull
conda activate doe
pip install -e .
```

If you can't use Git, you can use a ZIP instead, but updating takes more steps:

1. To set up, go to the project's GitHub page, choose **Code**, then **Download ZIP**, and unzip it. Then open a terminal in that folder and run `conda env create -f environment.yml`.
2. To update, download and unzip the new version, and keep your old folder for now. Copy `data/app_studies` from your old folder into the `data` folder of the new one.
3. Open a terminal in the new folder and run `conda activate doe`, then `pip install -e .`. This matters because the `doe` environment keeps using the code in your old folder until you do.
4. From now on, start the app from the new folder. If you made an icon, make it again from the new folder. Once your studies appear in step 1 of the app, you can delete the old folder.

The steps

The steps are listed in the sidebar:

Step	What you do
1. Define the study	Load an example or template, or enter factors and responses in the tables
2. Choose a design	State your objective and budget, compare the suggestions, and pick one. Download the run sheet and config
3. Enter results	Type results into the table, or upload the completed run sheet
4. Analyse	Choose a response, read the verdict, and explore the figures and tables. Change the model or transform if the checks suggest it
5. Find best settings	Get the best settings with predicted ranges, try your own settings, and download everything
Manual	This document

Adding more runs. Open **Add more runs to this experiment** in step 2. Choose how many runs you want, whether they'll be done in a separate session, and any repeats. You can also choose a fuller model and keep the new runs inside part of the range. **Suggest new runs** then shows the runs and what they'd add, and nothing changes until you select **Add these runs to my run sheet**. They're added to the end of the sheet, so download it again, then enter their results in step 3. See Adding more runs later for when and why to do this.

Saving your work on your own computer. The app saves your study automatically as you go, including half-filled tables, design settings, results and best settings. If you refresh or close the page, or the app stops, start it again and choose the study under **Carry on with a saved study** in step 1.

Studies are kept in `data/app_studies/`, with one folder for each study. Git ignores this folder, so back it up yourself. If a new design, an upload, simulated results or added runs change a sheet that has results, the previous version is kept in that study's `history` folder. To go back to one, load it with **Load saved study** in step 1.

Saving your work on a hosted copy. Nothing is saved on the server, and refreshing or closing the page loses your work, so use **Download my study** in the sidebar regularly. That one file holds everything, and you can load it in step 1 on any computer.

Existing data. To analyse an experiment you've already done, define its factors and responses in step 1, then upload the data in step 3. Column names must match the factor and response names exactly. You can skip step 2.

Sensitive data. If you're using a copy of the app hosted by someone else, check where it's hosted before uploading unpublished results. A copy running on your own computer keeps them there.

8. Using the scripts and config files

Scripts do the same calculations as the app, and suit anyone who prefers working with files or wants to automate studies.

Setting up

Follow the one-time setup in section 7, then run `conda activate doe` from the project folder.

Running a study

```
# Optional: get suggestions, or compare specific designs
python scripts/recommend_design.py configs/my_study.yaml --objective optimise --max-runs
30
python scripts/compare_designs.py  configs/my_study.yaml box_behnken central_composite
i_optimal:runs=24

# 1. Create the design. This writes data/my_study/runsheet.csv
python scripts/1_create_design.py configs/my_study.yaml

# ...do the experiments, and fill in the run sheet...

# 2. Analyse. Reports and figures go to results/my_study/
python scripts/2_analyse_results.py configs/my_study.yaml

# 3. Find the best settings
python scripts/3_optimise.py configs/my_study.yaml
```

To see a whole study first, without any lab work:

```
python scripts/1_create_design.py   configs/examples/hydrogel_bioink_simulated.yaml
python scripts/simulate_results.py  configs/examples/hydrogel_bioink_simulated.yaml
python scripts/2_analyse_results.py configs/examples/hydrogel_bioink_simulated.yaml
python scripts/3_optimise.py        configs/examples/hydrogel_bioink_simulated.yaml
```

Your results are protected: `1_create_design.py` won't replace a run sheet that already exists unless you add `--force`.

To add runs to a study you've already started:

```
# Add 8 runs in a new block, 2 of them repeating earlier runs
python scripts/add_runs.py configs/my_study.yaml --runs 8 --repeats 2

# Keep the new runs inside part of a factor's range, and aim for better estimates of
each effect
python scripts/add_runs.py configs/my_study.yaml --runs 6 --region pressure=60:90 --goal
effects

# The new runs will be done alongside the earlier ones, so don't add a block
python scripts/add_runs.py configs/my_study.yaml --runs 4 --same-session
```

The new runs go at the end of the run sheet, and your existing runs and results don't change. A copy of the previous run sheet is kept next to it. The runs are chosen for the model in your config file, so to go from screening to optimising, change `model` in the config to `quadratic` first. See Adding more runs later for when and why to do this.

Config file reference

Copy the closest file from `configs/templates/` and edit it. `python scripts/1_create_design.py --list` lists every design type.

Top-level settings

Setting	Meaning
`name`	The study's name, which also names its folders
`description`	Optional free text
`seed`	Fixes the random choices, so the same config recreates the same design
`factors`, `responses`	See below
`mixture_total`	What the mixture components add up to (default 1)
`constraints`	Limits on combinations, like `{expr: a + b, max: 16}`. Also accepts `min`
`design`, `model`	See below
`data_file`	Read results from this file instead of `data/<name>/runsheet.csv`. Used for existing data
`figure_format`	`svg` (default, with editable text), `png` or `pdf`
`simulation`	Made-up formulas for practising. See the worked example

Factors

Setting	Meaning
`name`	Letters, numbers and underscores, starting with a letter
`type`	`continuous`, `discrete`, `categorical` or `mixture`
`low`, `high`	Limits, for continuous and mixture factors
`levels`	Allowed values, for discrete and categorical factors
`units`	Optional, shown on figures

Responses

Setting	Meaning
`goal`	`maximise`, `minimise`, `target`, `in_range` or `none`
`low`, `high`	Limits. For maximise, low is the worst acceptable and high is as good as it gets. For minimise, it's the reverse. Measured values are used if these are left out
`target`	The ideal value, for `goal: target`
`importance`	How much it counts relative to the other responses (default 1)
`shape`	Bends the desirability curve: above 1 is stricter, below 1 is more lenient (default 1)
`transform`	`none`, `log` or `sqrt`

Design settings. Every design accepts `repeats` (run the whole design this many times), `replicate_runs` (extra repeats of individual runs), `blocks` and `randomise`.

`type`	Its own settings
`full_factorial`	`levels` (a number, or one per factor), `center_points`
`fractional_factorial`	`resolution` or `generators`, `center_points`
`plackett_burman`	`center_points`
`taguchi`	`array` (like `L9`), `levels`
`gsd`	`levels`, `reduction`, `center_points`
`box_behnken`	`center_points`
`central_composite`	`variant` (`face_centred`, `circumscribed` or `inscribed`), `alpha` (`rotatable` or `orthogonal`), `center_points`
`doehlert`	`center_points`
`latin_hypercube`, `sobol`	`runs`
`simplex_lattice`	`degree`, `check_points`
`simplex_centroid`	`check_points`
`extreme_vertices`	`edge_midpoints`, `check_points`
`crossed`	`mixture` and `process`, each with its own `type` and settings
`d_optimal`, `i_optimal`	`runs`, `candidate_levels`, `max_candidates`, `starts`
`existing`	Reads data you already have, and creates no design

Model settings

Setting	Options
`process`	`linear`, `pure_quadratic` (no interactions), `interaction`, `quadratic` or `cubic`
`mixture`	`linear`, `quadratic` or `special_cubic`
`mixture_process`	`linear` (a process effect can depend on the blend) or `none`
`terms`	A hand-written list, like `[a, b, a*b, a^2]`, which replaces the settings above
`reduce`, `alpha`	Turn on model reduction, and set the p-value above which terms are removed (default 0.10)

Folders

Folder	Contents
`app/`	The web app
`src/doekit/`	The toolkit's code
`scripts/`	The command-line steps
`configs/templates/`	Starting points to copy, one for each kind of study
`configs/examples/`	Worked examples with simulated results
`data/<study>/`	Run sheets, which hold your results. Git ignores this folder to keep them off GitHub, **so back them up yourself**
`data/app_studies/`	Studies saved automatically by the app. Git ignores this folder, **so back it up yourself**
`results/<study>/`	Reports and figures, which can be regenerated at any time
`docs/`	This manual (as Markdown and a PDF), and the hosting guide
`tests/`	Automatic checks of the toolkit
`archive/`	The original analysis script

9. Troubleshooting

What you see	What it means	What to do
"Your local changes... would be overwritten by merge"	`git pull` stopped because you've edited a file it needs to update. Nothing has changed	Copy your version of that file somewhere else if you want to keep it. Then run `git restore` followed by the file's path, and run `git pull` again
"No module named 'doekit.saved_study'"	The `doe` environment is still using the code in an old folder	Run `conda activate doe`, then `pip install -e .`, from the folder you start the app from
The icon says "doekit couldn't start"	The message says why, and the details are in `data/doekit_app.log` in the `doe` folder	If you moved the `doe` folder, run `python scripts/make_shortcut.py` again from its new place. Otherwise, start doekit from a terminal once to see the problem
On a Mac: "doekit can't be opened because it is from an unidentified developer"	The icon was copied from another computer, rather than made on yours	Make your own icon with `python scripts/make_shortcut.py`
"For better performance, install the Watchdog module"	The app only reloads more slowly when its code files change. Your results aren't affected	You can ignore it, or run `pip install watchdog` and restart the app
"This model can't be estimated"	The design doesn't have enough distinct runs to separate these terms	Use a simpler model, or a design with more runs
"These can't be told apart"	Two effects are confounded in this design	Use a higher-resolution design, or choose which term to keep
"A run sheet already exists"	A safeguard against losing results	Move or rename the old sheet, or use `--force` if you're sure
"...break your constraints"	Classical designs can't work around constraints	Use `i_optimal` or `d_optimal`
"...beyond your upper limits"	Limits cut corners off the blend region	Use `extreme_vertices` or `i_optimal`
"...has text instead of a number"	A result cell has words in it	Leave failed runs blank and explain in the notes
"The mixture components should add up to..."	A row doesn't match the total	Check that row, and that the total in the config is right
No settings meet all the limits	Your goals conflict within these ranges	Relax a limit, widen a factor's range, or accept a trade-off
Predicted R^2 is negative	The model predicts worse than using the average	Turn on model reduction, or check for an outlier
Everything looks non-significant	Effects are small compared with noise	Widen the ranges, reduce the noise, or add replicates
The best point sits on a limit	The optimum may lie outside the range you tested	Run a follow-up experiment that extends the range in that direction

10. What doekit doesn't do

Knowing the limits matters as much as knowing the features. These situations need other tools, or advice from a statistician.

- **Hard-to-change factors.** Some factors are impractical to reset for every run, such as a bath temperature that takes an hour to stabilise. People often respond by grouping those runs together. That's a **split-plot** experiment, and analysing it with ordinary regression overstates the confidence in the hard-to-change factors. doekit doesn't analyse split-plot designs, so ask for statistical advice if this applies.
- **Pass or fail responses, and counts.** The models assume a continuous response. "Printed or didn't" needs logistic regression instead. A viability percentage is usually fine when it stays away from 0 and 100%.
- **Non-independent measurements.** Several filaments from one print, or several wells from one batch, aren't independent runs. Average them into one result per run, and record the scatter separately.
- **Noise factors and robust design.** Taguchi's signal-to-noise analysis isn't included.
- **Definitive screening designs and Bayesian optimisation.** These are useful approaches, but they aren't included.
- **Factors outside your ranges.** Predictions only hold within the region you tested, and doekit warns when a prediction is an extrapolation.

11. Methods behind each step

You don't need this section to run a study. It shows what doekit actually does at each step, where each method comes from, and how to describe your methods in a paper. Short citations refer to section 13. Where the source is a textbook, it's a general reference for that topic, so check the relevant chapter before citing it. Settings shown are the defaults, and thresholds are the points at which the app raises a flag.

Creating the design

Design or step	Method	Done by	Source
Full factorial	Every combination of levels	pyDOE3	Montgomery
Fractional factorial	Regular two-level fractions, from the standard table of minimum-aberration generators. Resolution is calculated from the defining relation, and aliases are listed in the report	pyDOE3, with doekit's table	Box & Hunter (1961); Fries & Hunter (1980); Montgomery
Plackett-Burman	Two-level orthogonal main-effects plans	pyDOE3	Plackett & Burman (1946)
Taguchi	Orthogonal arrays, with factors assigned to columns with the right number of levels. Analysed by regression, not signal-to-noise ratios	pyDOE3, then doekit	Taguchi (1987)
Generalized subset	Balanced reduction of a multi-level factorial	pyDOE3	Surowiec et al. (2017)
Box-Behnken	Three-level design with no runs at the extreme corners. Above 5 factors it uses a pairwise construction, which needs more runs than the published tables	pyDOE3	Box & Behnken (1960)
Central composite	Two-level factorial core, plus star points and centre points. Face-centred by default	pyDOE3	Box & Wilson (1951)
Doehlert	Uniform shell: every difference between the corners of a regular simplex, scaled to your limits	doekit	Doehlert (1970)
Latin hypercube	Stratified sampling, improved by minimising centred discrepancy	SciPy	McKay et al. (1979)
Sobol	Scrambled low-discrepancy sequence	SciPy	Sobol' (1967)
Simplex lattice	Every blend with proportions in steps of 1/degree	doekit	Scheffé (1958)
Simplex centroid	Pure components and all equal-parts blends	doekit	Scheffé (1963)
Extreme vertices	Corners of the allowed region, found by setting all components but one to their limits, plus edge midpoints and the centre	doekit	McLean & Anderson (1966)
D-optimal, I-optimal	Fedorov exchange: repeatedly make the best single swap between a design run and a candidate run, from several starting designs (20, or fewer for large problems). D-optimal maximises det(X'X). I-optimal then minimises the average prediction variance across the candidates	doekit	Fedorov (1972); Goos & Jones (2011)
Candidate runs for optimal designs	Process factors on a grid of 3 levels (5 with constraints or cubic terms). Blends from corners, edge midpoints, the centre and a lattice. Up to 5,000 runs, sampled at random above that	doekit	Goos & Jones (2011)
Run count for optimal designs	Model terms + 4 runs to test lack of fit + 4 replicates	doekit	
Replicate runs	Repeats of the distinct runs with the highest leverage	doekit	
Blocks	Equal-sized blocks, improved by swapping pairs of runs to maximise det(X'X) with block effects included	doekit	Goos & Jones (2011)
Run order	Random permutation, within blocks if there are any	NumPy	Montgomery
Adding runs to a design	The same Fedorov exchange, with the existing runs' model terms held fixed in X'X, so only the new runs can be swapped. Runs with no result aren't counted. Block effects are included, and I-optimal averages over blocks. Repeats are chosen first, from the earlier runs with the highest leverage	doekit	Fedorov (1972); Goos & Jones (2011)

Judging a design before running it

These depend only on where the runs are placed, so they're given as multiples of the noise, which isn't known yet.

Measure	How it's calculated	Source
Standard error of a term	Square root of the term's diagonal entry in $(X'X)^{-1}$	Montgomery
Prediction error	Square root of $x'(X'X)^{-1}x$, across the candidate runs. The comparison plot sorts these values	Zahran et al. (2003)
Power	Two-sided t-test at $\alpha = 0.05$, using the noncentral t distribution. A low-to-high change of S × noise means a coefficient of S/2 in coded units. Not given for mixture terms	Montgomery
Entangled terms (VIF)	Diagonal entry of $(X'X)^{-1}$ × the term's corrected sum of squares. Flagged above 10, except for blend terms	Montgomery
D-efficiency	100 × det(X'X)^(1/p) / n, for p terms and n runs. In the design report	Goos & Jones (2011)
Effects that can't be separated	Model columns that are identical up to scale	

Fitting the model

Step	Method	Done by	Source
Fitting	Ordinary least squares	statsmodels	Montgomery
Process factors	Coded from −1 (low) to +1 (high)	doekit	Myers et al.
Default model orders	Linear, interaction, quadratic or cubic polynomials. The default depends on the design	doekit	Myers et al.
Categorical factors and blocks	Effect coding, so each coefficient is a difference from the overall mean. Blocks are fixed effects, and predictions average over them	doekit	Montgomery
Blends	Scheffé canonical polynomials with no intercept, on L-pseudo-components	doekit	Scheffé (1958); Cornell
Blends with process factors	Blend model + (component × process) terms + process-only higher-order terms	doekit	Cornell
Transforms	Fitted on the log or square-root scale. Predictions and intervals are converted back, so on the log scale they estimate the median	doekit	Box & Cox (1964)
Model reduction (optional)	Backward elimination: remove the least significant term until all have $p \le 0.10$, keeping the model hierarchical. Component terms always stay	doekit	Montgomery
Effect plots for blends	Cox direction: change one component while the others keep their proportions	doekit	Cornell

Checking the model

Check	Method	Flagged when	Source
Overall model	F-test of all model terms, excluding blocks	$p \geq 0.05$	Montgomery
Each term	Partial F-test comparing the model with and without the term (Type III). Component terms are tested together as "Linear mixture"	Not flagged, but shown in the table and plot	Montgomery; Cornell
R^2 and adjusted R^2	Calculated from the corrected total, including for models with no intercept	Not flagged	Montgomery
Predictive power	Predicted R^2, from the PRESS statistic (leave-one-out residuals)	Below 0, or more than 0.2 below adjusted R^2	Allen (1974)
Lack of fit	F-test against pure error. Pure error comes from fitting a separate mean to each distinct setting, plus any block effects	$p < 0.05$	Montgomery
Residual normality	Shapiro-Wilk test, with at least 8 runs and 3 spare	$p < 0.01$	Shapiro & Wilk (1965)
Constant variance	Spearman correlation between predicted values and absolute residuals, with at least 8 runs	Positive, with $p < 0.05$	
Transform	Box-Cox profile likelihood, for λ from -2 to 2 in steps of 0.1, with a 95% interval. Suggests none if the interval includes 1, then log if it includes 0, then square root if it includes 0.5	Interval excludes 1	Box & Cox (1964)
Run order	Durbin-Watson statistic, in run order	Below 1 or above 3	Durbin & Watson (1950)
Outliers	Externally studentised residuals	Size above 3.5	Montgomery
Influential runs	Cook's distance	Above 1	Cook (1977)
Curvature, in two-level designs	t-test comparing centre points with corners, using the scatter of the centre points	$p < 0.05$	Montgomery
No spare runs	Lenth's pseudo standard error and margin of error at $\alpha = 0.05$, with a half-normal plot	Effects above the margin stand out	Lenth (1989)
Prediction ranges	t-based 95% prediction interval for a single new run	Not flagged	Montgomery

Finding the best settings

Step	Method	Done by	Source
Scoring one response	Desirability function from 0 to 1, with `shape` as the exponent. Limits you don't set come from the lowest and highest values measured	doekit	Derringer & Suich (1980)
Combining responses	Weighted geometric mean, with `importance` as the weight	doekit	Derringer & Suich (1980)
Searching	Differential evolution, from 3 starting points for each combination of categorical and discrete levels (1 if there are more than 4 combinations)	SciPy	Storn & Price (1997)
Refining	Nelder-Mead simplex, from the best point found	SciPy	Nelder & Mead (1965)
Keeping blends on their total	The last component is whatever remains. Going past a limit or constraint incurs a penalty	doekit	
Choosing between equal settings	A very small preference for settings where the model has the lowest leverage	doekit	
Spotting extrapolation	Flagged when a setting's leverage is higher than that of any run in the study	doekit	Montgomery

Writing up your methods

You can adapt this wording, keeping only what applies to your study. Get your software versions by running `pip list` in the `doe` environment. Each version of doekit has its own DOI, listed on its Zenodo page.

> A [design type] with [n] runs, including [r] replicates, was generated with doekit [version] (Shields, [version DOI]) using pyDOE3 [version]. Runs were performed in random order[, in [b] blocks]. Each response was fitted by ordinary least squares in statsmodels [version] using a [model] model, with factors in coded units[, and with mixture components as Scheffé polynomials on pseudo-components][, after a [log] transformation][, and reduced by hierarchical backward elimination at p = 0.10]. Models were assessed by analysis of variance, lack-of-fit testing, adjusted and predicted R^2, and residual analysis. Responses were combined using desirability functions (Derringer & Suich, 1980), and the best settings were found by differential evolution (Storn & Price, 1997) in SciPy [version]. Predictions were tested with [c] independent confirmation runs.

Also report the results listed in step 10, and cite the original paper for your design type.

12. How doekit was checked

The automatic tests in `tests/` run with `pytest`. They include:

- **Published results.** Coefficients, standard errors, ANOVA tables and lack-of-fit tests match published analyses of these examples:
  - from Montgomery: the central composite chemical yield, the 2^4 filtration rate and its 2^{4-1} half fraction, and the battery life experiment, with its categorical factors and with temperature as a curved numeric factor
  - from Myers, Montgomery and Anderson-Cook: a chemical reaction run in two blocks, analysed both before and after the second block was added
  - from Cornell: the yarn elongation mixture

  Best settings match a published stationary point and a published two-response desirability optimum.
- **Certified accuracy.** On NIST's Statistical Reference Datasets for regression, the results agree with NIST's certified values to at least 7 significant digits, and usually 13 to 15. This includes the Longley and Filip datasets, which are notoriously difficult.
- **The original analysis.** The melt electrowriting model matches the original analysis script to within rounding error. This check only runs where the original data is available, because that data isn't in the public repository.
- **The design algorithms.** The optimal design search finds the same answer as testing every possibility on small problems, and reproduces a known theoretical result for mixtures. Every tabulated fractional factorial's resolution is recalculated, and Box-Behnken and rotatable central composite designs match their published tables.
- **Adding runs.** With existing runs held fixed, the search still finds the same answer as testing every possibility on small problems. Existing runs and results stay unchanged, a shift between sessions doesn't change the estimated effects, and a two-level design with added runs recovers a known curved formula exactly. Given the first block of the published two-block reaction experiment, it adds exactly the axial points of the textbook face-centred central composite design.
- **Known truths.** On simulated data, the analysis recovers the formulas behind it, and 95% prediction intervals contain about 95% of new runs.
- **The whole workflow.** Every template and example runs from design through to best settings, and the web app is clicked through automatically.

These tests show that the calculations are right. They can't show that a particular experiment was well planned, which is why steps 1, 4 and 9 of the procedure matter.

13. References

Books

- Cornell, J.A. *Experiments with Mixtures.* Wiley.
- Fedorov, V.V. (1972). *Theory of Optimal Experiments.* Academic Press.
- Goos, P., Jones, B. (2011). *Optimal Design of Experiments: A Case Study Approach.* Wiley. A readable guide to D-optimal and I-optimal designs.
- Montgomery, D.C. *Design and Analysis of Experiments.* Wiley. The standard introduction.
- Myers, R.H., Montgomery, D.C., Anderson-Cook, C.M. *Response Surface Methodology.* Wiley.
- Taguchi, G. (1987). *System of Experimental Design.* UNIPUB/Kraus International.

Papers

- Allen, D.M. (1974). The relationship between variable selection and data augmentation and a method for prediction. *Technometrics* 16(1), 125–127.
- Box, G.E.P., Behnken, D.W. (1960). Some new three level designs for the study of quantitative variables. *Technometrics* 2(4), 455–475.
- Box, G.E.P., Cox, D.R. (1964). An analysis of transformations. *Journal of the Royal Statistical Society B* 26(2), 211–243.
- Box, G.E.P., Hunter, J.S. (1957). Multi-factor experimental designs for exploring response surfaces. *Annals of Mathematical Statistics* 28(1), 195–241.
- Box, G.E.P., Hunter, J.S. (1961). The 2^(k−p) fractional factorial designs, part I. *Technometrics* 3(3), 311–351.
- Box, G.E.P., Wilson, K.B. (1951). On the experimental attainment of optimum conditions. *Journal of the Royal Statistical Society B* 13(1), 1–38.
- Cook, R.D. (1977). Detection of influential observation in linear regression. *Technometrics* 19(1), 15–18.
- Derringer, G., Suich, R. (1980). Simultaneous optimization of several response variables. *Journal of Quality Technology* 12(4), 214–219.
- Doehlert, D.H. (1970). Uniform shell designs. *Applied Statistics* 19(3), 231–239.
- Durbin, J., Watson, G.S. (1950). Testing for serial correlation in least squares regression: I. *Biometrika* 37(3/4), 409–428.
- Fries, A., Hunter, W.G. (1980). Minimum aberration 2^(k−p) designs. *Technometrics* 22(4), 601–608.
- Lenth, R.V. (1989). Quick and easy analysis of unreplicated factorials. *Technometrics* 31(4), 469–473.
- Lenth, R.V. (2009). Response-surface methods in R, using rsm. *Journal of Statistical Software* 32(7), 1–17.
- McKay, M.D., Beckman, R.J., Conover, W.J. (1979). A comparison of three methods for selecting values of input variables in the analysis of output from a computer code. *Technometrics* 21(2), 239–245.
- McLean, R.A., Anderson, V.L. (1966). Extreme vertices design of mixture experiments. *Technometrics* 8(3), 447–454.
- Nelder, J.A., Mead, R. (1965). A simplex method for function minimization. *The Computer Journal* 7(4), 308–313.
- Plackett, R.L., Burman, J.P. (1946). The design of optimum multifactorial experiments. *Biometrika* 33(4), 305–325.

- Scheffé, H. (1958). Experiments with mixtures. *Journal of the Royal Statistical Society B* 20(2), 344–360.
- Scheffé, H. (1963). The simplex-centroid design for experiments with mixtures. *Journal of the Royal Statistical Society B* 25(2), 235–251.
- Shapiro, S.S., Wilk, M.B. (1965). An analysis of variance test for normality (complete samples). *Biometrika* 52(3/4), 591–611.
- Sobol', I.M. (1967). On the distribution of points in a cube and the approximate evaluation of integrals. *USSR Computational Mathematics and Mathematical Physics* 7(4), 86–112.
- Storn, R., Price, K. (1997). Differential evolution: a simple and efficient heuristic for global optimization over continuous spaces. *Journal of Global Optimization* 11(4), 341–359.
- Surowiec, I., et al. (2017). Generalized subset designs in analytical chemistry. *Analytical Chemistry* 89(12), 6491–6497.
- Zahran, A., Anderson-Cook, C.M., Myers, R.H. (2003). Fraction of design space to assess prediction capability of response surface designs. *Journal of Quality Technology* 35(4), 377–386.

Reference data used to check doekit

- Kuhn, M. desirability: function optimization and ranking via desirability functions. R package, with a vignette that works through Myers and Montgomery's two-response example. https://cran.r-project.org/package=desirability
- National Institute of Standards and Technology. Statistical Reference Datasets: linear least squares regression. https://www.itl.nist.gov/div898/strd/

Software. doekit uses pyDOE3 for classical designs, statsmodels for regression, SciPy for search and sampling, mpltern for blend plots, and Streamlit for the web app. To cite the main ones:

- Shields, F. doekit: design, analyse and optimise experiments. Zenodo. https://doi.org/10.5281/zenodo.23982051
- Seabold, S., Perktold, J. (2010). statsmodels: econometric and statistical modeling with Python. *Proceedings of the 9th Python in Science Conference.*
- Virtanen, P., et al. (2020). SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods* 17, 261–272.